1. Introduction to PPO and SAC

Proximal Policy Optimization (PPO) [1] and Soft Actor-Critic (SAC) [2] are two widely used deep reinforcement learning algorithms which are introduced in the late 2010s. These algorithms are use for continous control but they differ in how they use the *data*. PPO is an on-policy method, which means that each policy iteration update is computed using tragectories collected by the current policy, and when the policy is updated all previously gathered data is discarded. SAC, on the other hand, is an off-policy method, it stores the previous transitions in a replay buffer, meaning that it can continue to learn from older transitions stored in the buffer, which usually makes it much more sample efficient.

PPO is a gradient method that seeks to take the largest possible improvement step without destabilising the policy. It was developed as a practical improvement over Trust Region Policy Optimization (TRPO) @trpo, which enforced a KL divergence constraint on the policy updates. PPO replaces the hard trust-region style of KL divergence with a clipped surrogate objective.

Before PPO updates the policy, we must estimate how much better each action was compared to the baseline. We implement this via Generalised Advantage Estimation (GAE), which iterates backward through a collected rollout. At each timestep t, we compute the TD error:

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_t) - V(s_t) \tag{1}$$

where d_t is 1 if the episode terminated at step t (masking the bootstrap when there is no valid next state). The advantage is then accumulated via recursively going back the timesteps:

$$\hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1} \tag{2}$$

with $\hat{A}_T = 0$ at the rollout boundary. The parameter λ controls the bias-variance tradeoff in advantage estimation: at $\lambda = 0$, estimates reduce to one-step TD errors i.e. low variance but biased because they rely on the value function's accuracy. At $\lambda = 1$, estimation becomes equivalent to Monte Carlo returns, unbiased but high variance due to summing over full trajectories. The return targets used to train the critic are then $R_t = \hat{A}_t + V(s_t)$.

The core update mechanism relies on the ratio between the updated and the old policies:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{3}$$

Though we implement this a bit differently; we compute in log-space for numerical stability:

$$\log r_t(\theta) = \log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid s_t) \tag{4}$$

then exponentiate to obtain Equation 3. When this ratio is close to 1, the new policy behaves similarly to the old one. PPO prevents this ratio from deviating too far by clipping it:

$$L^{\text{CLIP}}(\theta) = \frac{1}{|\mathcal{B}|}\sum_{t\in\mathcal{B}} \max\Big(-\hat{A}_t \cdot r_t(\theta), -\hat{A}_t \cdot \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\Big) \tag{5}$$

Note that this is equivalent to the original paper's formulation $\min\Big(r_t\hat{A}_t, \text{clip}(r_t)\hat{A}_t\Big)$ after negation for gradient descent. The intuition here is simple: if the advantage is positive, i.e. the action was good, we would want to increase its probability, but only up to a factor of $(1+\epsilon)$. If the advantage is negative, we do the same but clip at $(1-\epsilon)$. The clipping creates a "pessimistic" lower bound on the policy improvement i.e. the gradient vanishes once the ratio leaves the trusted interval, preventing the catastrophic failures observed with naive policy gradient methods.

The critic $V_\phi(s)$ is trained alongside the policy by minimising MSE against the GAE return targets:

$$L^{\text{VF}}(\phi) = \frac{1}{|\mathcal{B}|}\sum_t \big(V_\phi(s_t) - R_t\big)^2 \tag{6}$$

This lets the critic learn a smoother estimate of the expected future return, which in turn improves the quality of the advantage estimates used by the actor. In practice, PPO collects a rollout of 2048

steps, computes GAE over the entire buffer, then performs 10 epochs of minibatch gradient descent, recomputing policy log-probabilities on each minibatch and updating both actor and critic. The importance-sampling ratio corrects for the fact that by later epochs, the policy has drifted from the one that collected the data, and the clipping mechanism ensures this drift remains bounded.

Soft Actor Critic (SAC) is an off-policy actor-critic RL algorithm based on the maximum entropy reinforcement leanring framework. The soft actor-critic algorithm incorporates three key ingredients: an actor-critic architecture with separate policy and value function networks, an off-policy formulation that enables reuse of previously collected data for efficiency, and entropy maximization to enable stability and exploration [2]. It consists of an actor trying arrive at an optimal policy and a critic that evaluates the generated policy. Both actor critic tend to get better with training, critic building more accurate estimations of state value and Q functions and the actor generating policies with higher returns. The algorithm maintains four different set of weights as described ahead: ψ for a soft value function approximator, $\boldsymbol{V}_{\psi}$ that essentially estimates state value functions of various states, θ for Q value network or critic, $\boldsymbol{Q}_{\boldsymbol{\theta}}$, that essentially estimates Q-function values for various state action pairs, ϕ for the policy network or actor,$\boldsymbol{\pi}_{\phi}$, that generates policies and finally $\overline{\psi}$ which represents a target value function $\boldsymbol{V}_{\overline{\psi}}$ and is updated as a moving average of the weights of the soft value function approximator V_{ψ}. The following loss functions were proposed in the original paper and the weights are updated by minimising over the mentioned loss functions.

$$J_V(\psi) = E_{s_t \sim D}\left[\frac{1}{2}\Big(V_\psi(s_t) - E_{a_t \sim \pi_\phi}[Q_\theta(s_t, a_t) - \log \pi(a_t|s_t)]\Big)^2\right] \tag{7}$$

$$\begin{aligned} J_Q(\theta) &= E_{(s_t,a_t)\sim D}\left[\frac{1}{2}\Big(Q_\theta(s_t, a_t) - \hat{Q}(s_t, a_t)\Big)^2\right] \text{ where,} \\ \hat{Q}(s_t, a_t) &= r(s_t, a_t) + \gamma E_{s_{t+1}\sim p}\left[V_{\overline{\psi}}(s_{t+1})\right] \end{aligned} \tag{8}$$

$$J_\pi(\phi) = E_{s_t \sim D, \varepsilon_t \sim N}\left[\log \pi_\phi\big(f_\phi(\varepsilon_t; s_t)|s_t\big) - Q_\theta(s_t, f(\varepsilon_t; s_t))\right] \tag{9}$$

$$\overline{\psi} \leftarrow \tau\psi + (1-\tau)\overline{\psi} \tag{10}$$

However in the code implementation we don't maintain a separate soft value network and directly train ψ on critic losses replacing θ in Equation 8. Essentially Equation 8 can be rewritten as the following:-

$$\begin{aligned} J_Q(\psi) &= E_{(s_t,a_t)\sim D}\left[\frac{1}{2}\Big(Q_\psi(s_t, a_t) - \hat{Q}(s_t, a_t)\Big)^2\right] \text{ where,} \\ \hat{Q}(s_t, a_t) &= r(s_t, a_t) + \gamma E_{s_{t+1}\sim p}\left[V_{\overline{\psi}}(s_{t+1})\right] \end{aligned} \tag{11}$$

The target netwrok weights $\overline{\psi}$ are now calculated directly as a moving average of the critic weights.

A neat little trick that was proposed in the original paper and is also a part of the code implementation that is worth noting is that, we actually train two sets of Q-function network weights $\{\psi_1, \psi_2\}$ that are trained independently to mitigate positive bias. The minimum of the two Q-functions is then utilised in Equation 7 and Equation 9.

SAC was an improvement over previously proposed RL methods in terms of sample efficiency as it is an off-policy method and stability to convergence and sensitivity to hyperparameters which was notoriously tough to achieve with off-policy model free methods.

2. Relevance to Robotics

Robotics problems typically involve continuous state and action spaces, noisy observations, and a strong need for stable learning that makes classical RL approaches impractical. PPO and SAC address several of these, which has led to them becoming baseline algorithms in robotics research.

Both methods natively operate in continuous state and action spaces. PPO parameterises a Gaussian policy from which actions are sampled, while SAC uses a squashed Gaussian to produce bounded

continuous actions. This is fundamental to robotics where the agent must output real-valued torques, joint angles, or velocities rather than choosing from a discrete set.

PPO is comparatively robust to catastrophic policy updates because of the clipped objective, and is straightforward to implement i.e. there is a single optimiser, no replay buffer, no target networks. This simplicity also makes it easy to parallelise across many simulation instances, making PPO most feasible in simulation-driven workflows (sim-to-real pipelines) where data generation is cheap.

SAC is especially appealing for real-world robotics, where data collection is expensive and limited. Being an off-policy method, it reuses all past experience via a replay buffer, making it substantially more sample efficient. SAC's entropy-regularised objective also encourages the policy to remain stochastic, reducing over-reliance on any single observation and providing inherent robustness to the sensor noise present in physical systems.

3. Environment Description

We chose to train in the Ant-v4 and HalfCheetah-v4 environments in MuJoCo gymnasium [3]. These were chosen because the represent different complexities of the same locomotion goal. This also makes them directly comparable for benchmarking PPO and SAC.

The **HalfCheetah** is a 2-dimensional robot consisting of 9 body parts and 8 joints connecting them (including two paws). The goal is to apply torque to the joints to make the cheetah run forward (right) as fast as possible, with a positive reward based on the distance moved forward and a negative reward for moving backward. The cheetah's torso and head are fixed, and torque can only be applied to the other 6 joints over the front and back thighs (which connect to the torso), the shins (which connect to the thighs), and the feet (which connect to the shins).

The **Ant** is a 3D quadruped robot consisting of a torso (free rotational body) with four legs attached to it, where each leg has two body parts. The goal is to coordinate the four legs to move in the forward (right) direction by applying torque to the eight hinges connecting the two body parts of each leg and the torso (nine body parts and eight hinges).

4. PPO Hyperparameter Tuning

We chose to tune clip coefficient (ϵ) and λ (in GAE) as these affect the algorithm's working to its core.

The clip coefficient controls the policy update size, this effectively dictates how quickly or slowly the policy is moving towards the optimal. Having the correct step size is important because if the step size is too small then the policy would take a lot of timesteps to reach the optimal or else if the step size is to big the policy would end up overshooting the optimal, both cases result in the formulation of a poor policy.

Tuning lambda controls the bias-variance trade off in estimating the advantage term. As lambda moves from 0 to 1 it cause the shift in advantage estimation being carried out as TD(0) at $\lambda = 0$ (high bias) and as Monte Carlo method at $\lambda = 1$ (high variance). Temporal Difference introduces bias as it bootstraps value estimates from the immediate next step. Monte Carlo methods are unbiased but inherently show high variance, this originates from variance being accumulated over the length of the episodes. Hence, requires the agent to be run on many episodes for the value estimates to reliably converge.

The following values of ϵ were chosen: $\epsilon \in \{0.1, 0.2, 0.3\}$. We have explored values around the published default [1] ($\epsilon = 0.2$) testing change in agent behaviour under a transition from a strict ($\epsilon = 0.1$) to a permissive ($\epsilon = 0.3$) policy update constraint. For λ the following values were chosen: $\lambda \in \{0.9, 0.95, 1.00\}$. Similar strategy of exploring around the published defualt [1] is followed, testing change in agent behaviour as advatage estimation moves from pure monte carlo at $\lambda = 1$ to sligthy towards TD(0) at $\lambda = 0.90$.

	$\lambda = 0.90$	$\lambda = 0.95$	$\lambda = 1.00$
$\epsilon = 0.1$	1471.80	1378.69	1657.81
$\epsilon = 0.2$	**4449.39**	2706.83	2397.13
$\epsilon = 0.3$	1131.88	**4568.06**	150.90

Table 1 : Average episodic return in HalfCheetah-v4 environment with different γ and τ configurations averaged over 10 episodes using PPO algorithm

	$\lambda = 0.90$	$\lambda = 0.95$	$\lambda = 1.00$
$\epsilon = 0.1$	2498.17	1419.31	11.57
$\epsilon = 0.2$	2277.72	3049.39	107.12
$\epsilon = 0.3$	**3204.60**	507.76	72.97

Table 2 : Average episodic return in Ant-v4 environment with different γ and τ configurations averaged over 10 episodes using PPO algorithm

5. SAC Hyperparameter Tuning

For the purpose of hyperparameter tuning, γ, the reward discount and tau, the polyak averaging constant were tuned.

γ plays a pivotal role in dictating the agents behaviour. It controls, to describe it intuitively, the patience of the agent. Having a larger γ forces to the agent to make more long sighted decisions that is give more weight to future rewards, on the other hand a smaller γ forces the agents to make short sighted decisions.

τ, or the polyak averaging constant, is used in calculating a moving average of the value network weights directly contribute to Q-function leanring. It weighs the contribution of current value network weights and target value network weights (previous moving average estimate). Essentially controlling the influence of immediate changes encountered as part of learning.

For hyperparameter tuning the number of training steps were reduced to 450k from 1000k(1 million).

The following values for γ were chosen: $\gamma \in \{0.90, 0.95, 0.99\}$. Here we have tested the change that arises from agressive discounting($\gamma = 0.90$) to long horizon planning ($\gamma = 0.99$). For τ we chose: $\tau \in \{0.05, 0.001, 0.005\}$. Here we have explored towards the upper range of τ relative to the published default [2] because exploring even lower values of τ on a limited training budget would be uninformative.

	$\gamma = 0.90$	$\gamma = 0.95$	$\gamma = 0.99$
$\tau = 0.005$	2294.81	2170.51	8556.36
$\tau = 0.01$	2371.91	8707.92	8552.37
$\tau = 0.05$	2385.95	2174.07	**9418.79**

Table 3 : Average episodic return in HalfCheetah-v4 environment with different γ and τ configurations averaged over 10 episodes using SAC algorithm

	$\gamma = 0.90$	$\gamma = 0.95$	$\gamma = 0.99$
$\tau = 0.005$	652.43	2143.56	
$\tau = 0.01$	892.43	1616.85	5016.65
$\tau = 0.05$	700.09	2290.87	3480.42

Table 4 : Average episodic return in Ant-v4 environment with different γ and τ configurations averaged over 10 episodes using SAC algorithm

6. Results and Comparison

We found a general trend of decreasing performance while shifting from HalfCheetah-v4 environment to Ant-v4 environment. This can be attributed to an increase in complexity for achieving locomotion. [Ant-v4 more complex because more joints to control $6 \rightarrow 8$, there is also a complexity of maintaining a heading or building a heading agnostic locomotion strategy.]. SAC consistently outperformed PPO across both environments; this has been discussed in detail later in this section.

$\lambda = 0.95$ & $\epsilon = 0.3$ We got the best performance from **PPO** for **HalfCheetah-v4** environment using $\boldsymbol{\{\lambda = 0.95, \epsilon = 0.3\}}$ giving an average episodic return of **4568.06** and for **Ant-v4** environment using $\boldsymbol{\{\lambda = 0.9, \epsilon = 0.3\}}$ giving an average episodic return of **3204.60**. We can infer that a more permissive policy update constraint was required because of having a limited training budget. We notice a shift towards Temporal Difference approximations of advantage showing better performance dk y.

- comparatively SAC gave better performance than PPO

Even though these methods work generally well and accomodate the above mentioned challenges, it is important to note that they individually work best for certain cases. PPO is feasible and effective in simulation-driven robotics workflows, where large amounts of data can be generated cheaply. In contrast, SAC is more feasible for real-world robotics applications. Its sample efficiency, robustness to noise, and ability to learn from off-policy data make it better suited for physical systems with limited interaction budgets, however it is tough to tune and is rather sensitive to hyperparameter tuning.

- SAC
  - **high γ**: locomotion tasks are long horizon planning tasks
  - **high τ**: performed better with incorporating immediate changes
- PPO
  - **high ϵ**: permissive policy update constraint
  - **mid λ**: exact monte carlo ($\lambda = 1$) is not good should just lean towards monte carlo

7. Proposed Robotics Task

- study POMDP

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *CoRR*, 2017, [Online]. Available: http://arxiv.org/abs/1707.06347

[2] T. Haarnoja *et al.*, "Soft Actor-Critic Algorithms and Applications," *CoRR*, 2018, [Online]. Available: http://arxiv.org/abs/1812.05905

[3] M. Towers *et al.*, "Gymnasium: A Standard Interface for Reinforcement Learning Environments." [Online]. Available: https://arxiv.org/abs/2407.17032